Matrioshkas

Una solución:

Observemos que en una solución óptima tenemos una matrioska exterior y dentro la combinación con más muñecas que caben dentro de esta muñeca exterior.

Para poder derivar una recurrencia correcta tenemos que establecer un orden que nos permita garantizar que las muñecas que caben dentro de la muñeca i aparecen posteriormente. Para conseguir esta propiedad ordenamos por el valor decreciente de h_j y aplicamos w_i como criterio de desempate. (valor decreciente) con coste $O(n \log n)$.

Así si la muñeca i cabe dentro de la muñeca j tenemos $j \leq i$ aún cuando puede haber muñecas con $k > j$ que no caben dentro de la muñeca i.

Utilizando esta propiedad y la estructura de suboptimalidad tenemos la siguiente recurrencia.

$N(i)$ = número máximo de muñecas que se pueden apilar dentro de la muñeca i, incluyendo muñeca i.

(~~Estos datos~~

$N(i) = 0$ si $\forall_{n \geq j > i}$ j no cabe dentro de i

$$N(i) = \max_{\substack{n \geq j > i \\ h_j < h_i \\ w_j < w_i}} \{ 1 + N(j) \} \qquad (*)$$

La recurrencia se puede implementar con un recorrido en tabla con coste $O(n^2)$

Finalmente podemos obtener ~~calcula~~ el tamaño máximo posible calculando $\max_{1 \leq i \leq n} M[i]$ con coste $O(n)$

Para obtener la composición de una matrioshka con número máximo de muñecas añadimos los punteros necesarios para guardar la decisión tomada en $(*)$.

El coste total del algoritmo en $O(n^2)$.

MenjaBe

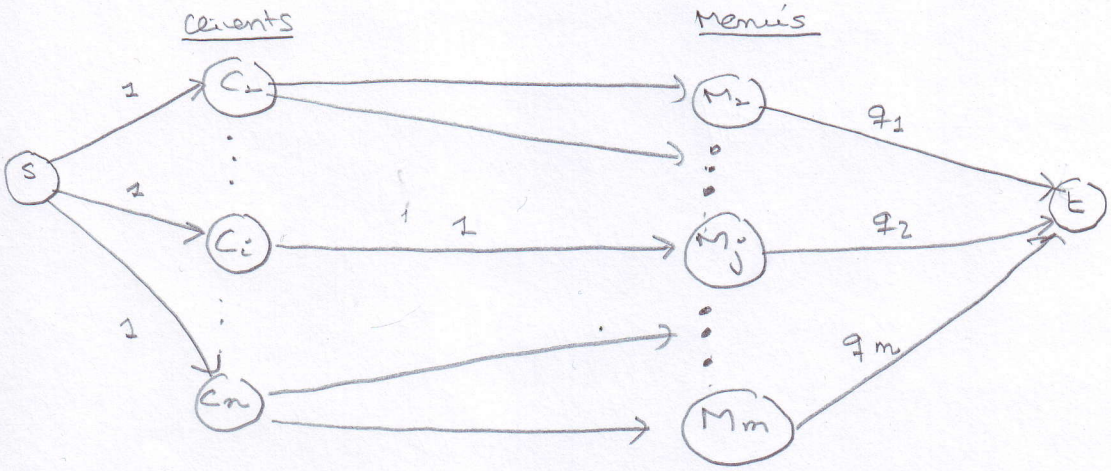
Resoldrem el problema via Max-Flow.

Definim la xarxa següent $G=(V,E)$ amb capacitats $c: E \to \mathbb{Z}^+ \cup \{0\}$

* <u>Vértex</u> $V = \{s, t, c_1, \dots, c_n, M_1, \dots, M_m\}$

  on $\{c_1, \dots, c_n\}$ representen el conjunt dels n clients

  i $\{M_1, \dots, M_m\}$ " " " dels m menus



$(c_i, M_j) \in E \iff M_j \in A_i$ (menú j és a la llista de preferències del client c_i

* <u>Arestes i capacitat</u>:

  i) Per cada client c_i, $1 \le i \le n$,

  $(s, c_i) \in E$ i $c(s, c_i) = 1$

  ii) Per cada client c_i per $1 \le i \le n$ i per cada menú M_j

  tal que $M_j \in A_i$ (llista de preferències del client)

  $(c_i, M_j) \in E$ i $c(c_i, M_j) = 1$

  iii) Per cada menu M_j, $1 \le j \le m$

  $(M_j, t) \in E$ i $c(M_j, t) = q_j$ (quantitat

  de menús de tipus M_j que MenjaBe pot servir

- En cop construïda la xarxa, apliquem un algorisme de Max-Flow de manera que si f correspon si $f: E \to \mathbb{N}$ és el flux màxim calculat per l'algorisme podem definir l'assignació òptima per a MenjaBe

$$assig: \underbrace{\{C_1, \dots, C_n\}}_{\text{clients}} \longrightarrow \underbrace{\{M_1, \dots, M_m\}}_{\text{menus}} \cup \{\text{"val"}\}$$

- $f(s, C_i) = 1 \Rightarrow \exists j \; M_j \in A_i \quad f(C_i, M_j) = 1$

  En aquest cas definim $assig(C_i) = M_j$

  el cost per a MenjaBe, $cost(C_i) = 0$

- $f(s, C_i) = 0 \Rightarrow \forall j \; M_j \in A_i \quad f(C_i, M_j) = 0$

  En aquest cas $assig(C_i) =$ "val"

  el cost per a MenjaBe, $cost(C_i) = 5$

- $f(M_j, t) \leq q_j \Rightarrow$ Mai se serveixen més menus dels que es disposa

- L'algorisme de Max-Flow maximitza $|\{C_i \mid f(s, C_i) = 1\}|$

  i per tant maximitza $|\{C_i \mid cost(C_i) = 0\}|$

  i això vol dir que minimitza $|\{C_i \mid cost(C_i) = 5\}|$

  minimitza el nombre de vals que ha de repartir

Així tenim demostrada la correctesa

Eficiència

Quin algorisme seria més aconsellat d'entre Ford-Fulkerson i Edmons-Karp?

Notem que $|V| = n+m+2$

$$|E| = n + \sum_{i=1}^{n} |A_i| + m = O(nm)$$

- Ford-Fulkerson: $O(|E||f^*|) = O(n^2 \cdot m)$ $\quad (|f^*| \leq n)$
- Edmons-Karp: $O(|V||E|^2) = O(n^3m^2 + n^2m^3)$

En aquest cas el més eficient seria Ford-Fulkerson i caldria afegir el temps necessari per calcular l'assignació a partir del fluxo.

En total $T(n,m) = O(n^2 \cdot m)$

$\blacksquare$

Menja Bé

apartat b) Examen Parcial

1.- Construir una xarxa com l'apartat a) considerant només els clients que tenen una llista de preferències (Ci no menú sorpresa)

Calcular el Max.Flow $f^*_{nosorpresa}$

minimitzant així $|\{C_i \mid A_i \neq \{sorpresa\} \wedge assig(C_i) = \text{"val"}\}|$

és a dir minimitzant el nombre de vals de 5 euros dintre el conjunt $\{C_i \mid A_i \neq \{sorpresa\}\}$

2. Calculem $\varphi = \sum_{i=1}^{m} q_j - |f^*_{nosorpresa}|$ menús que no han estat assignats encara

$S = |\{C_i \mid A_i = \{sorpresa\}\}|$ nombre de clients que desitgen menú sorpresa

si $\varphi \geq S$ aleshores assignem un menú sorpresa als els clients que volien menú sorpresa cost mínim per $|\{C_i \mid A_i = \{sorpresa\}\}|$: $3 \times S$

Si $\varphi < S$ aleshores assignem tots els menus sorpresa possible, en total φ, i assignem un val a lo roste, en total $S-\varphi$ i aleshores el cost mínim per $|\{C_i \mid A_i = \{sorpresa\}\}|$:

$3 \times S + 5 \times (S-\varphi)$

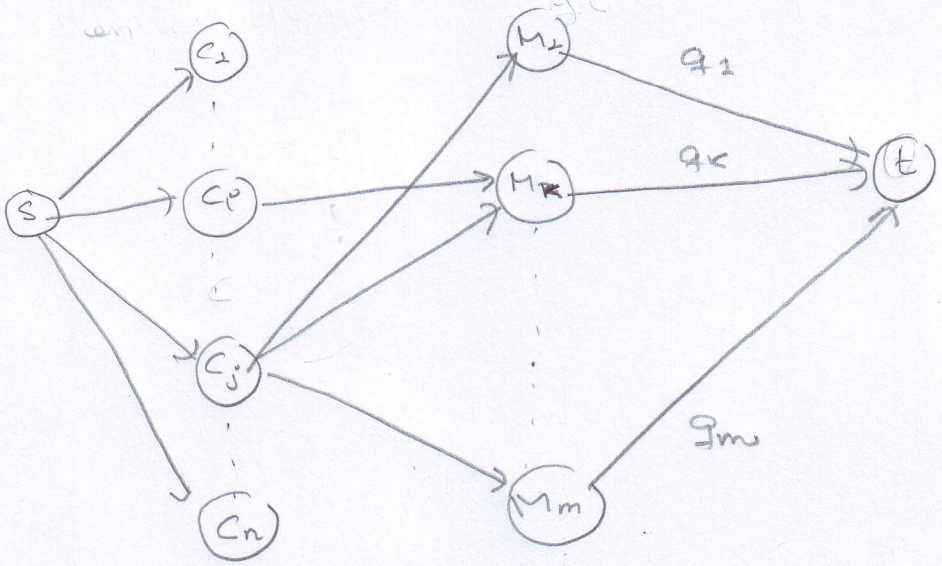
apartat b) Continuació

3.- No es poden "barejar" menus

clients c_i amb $A_i \subseteq \{M_1, \dots, M_m\}$ amb

clients c_j amb $A_j = \{$menu-sorpresa$\}$

Si a la xarxa tingussim de matèix temps aquests 2 tipus de clients, el client "sorpresa" accepta qualsevol menú.



Per exemple, si $q_k = 1$ i

$A_i = \{M_k\}$

$A_j = \{$sorpresa$\}$ $\quad (c_j, M_k) \in E \quad \forall k \quad 1 \le k \le m$

maximitzar fluxe $\not\Rightarrow$ minimitzar cost

si $f(c_i, M_k) = 1 \Rightarrow$ cost $= 0$

si $f(c_j, M_k) = 1 \Rightarrow$ cost $= 3$?

Amb això tenim la correcció.

- <u>Eficiència de l'algorisme</u> proposat

segueix l'anàlisi de l'apartat a) afegint l'assignació de menus sorpresa ($O(n)$ addicional)

$T(n,m) = O(n^2 m)$

Cognoms, Nom Data

Exercici 4 (4 punts)

(a) (0.5 punts) L'ordenació RADIX funciona correctament quan, per a ordenar els dígits, utilitzem qualsevol algorisme d'ordenació que sigui correcte.

Solució. Fals. L'algorisme d' ordenació ha de ser estable.

(b) (0.5 punts) Donat un vector d' enters $A[1, \ldots, n]$, la complexitat d'ordenar-lo utilitzant comptatge és $O(n)$.

Solució Fals. Depens de la grandària de cada enter.

(c) (0.5 punts) Suposem que tenim n intervals $I_i = (e_i, d_i)$, $1 \leq i \leq n$, i que cada interval té associat un pes w_i. Volem trobar un conjunt S d'intervals que no es sobreposen i que maximitzi $\sum_{i \in S} w_i$. Considereu el següent algorisme: Comencem amb el conjunt T que conté tots els intervals. A cada pas, seleccionem de T un interval I_j amb màxim w_j i ho afegim a S. Després eliminem de T l'interval I_j i tots els intervals a T que es sobreposin amb I_j. Repetim fins a no poder seleccionar-ne més. Quin tipus d'esquema algorísmic és aquest algorisme? Es cert que aquest algorisme produeix una solució òptima del problema?

Solució: Es un greedy i com hem vist a classe no produeix sempre una solució òptima.

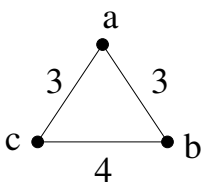
(d) (0.5 punts) Si utilitzem l'algorisme de Huffmann per a comprimir un text format per n símbols que apareixen amb freqüències $f_1, f_2, f_3, \ldots, f_n$ respectivament, quina és la màxima longitud de compressió d'un símbol que podem obtenir? Doneu un algorisme per obtenir freqüències on es compleixi aquesta condició per a un valor de n donat.

Solució: La màxima longitud és $n-1$. Si agafem $f_i = \frac{1}{2^i}$, per $1 \leq i \leq n-1$ i $f_n = \frac{1}{2^{(n-1)}}$, el últim i el penultim símbol tindran longituds $n-1$.

Podem derivar facilemen l'agorisme que initializa $F[1] = 1/2$ i per $1 < i < n$, fa $F[i] = F[i-1]/2$. Finalment $F[n] = F[n-1]$.

(e) (0.5 punts) Sigui $G = (V, E)$ un graf amb pesos $w(E)$ i sigui T un arbre d'expansió amb cost mínim (MST) a G. Aleshores el camí de pes mínim a G entre dos vèrtexs v_1 i v_2 ha de ser també un camí mínim a T.

Solució FALS, considereu la figura de sota, T conté arestes (c, a) i (a, b) però el camí mínim de c a b és l'aresta (c, b).

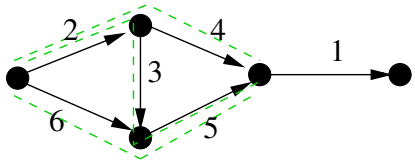


(f) (0.5 punts) Donat un graf no dirigit $G = (V, E)$, definim el seu *diàmetre* d com la màxima distància entre qualsevol parell de vèrtexs de G. Doneu un algorisme, el més eficient possible, que amb entrada un graf no dirigit qualsevol G amb n vèrtexs i m arestes, calculi el diàmetre de G.

Solució: Utilitzant l'algorisme de Bernard-Floyd-Warshall podem obtenir les distanciès entre qualsevol parell de punts. Després hem de calcular el màxim. El cost és $O(n^3)$ tant si és dens com si és espars.

(g) (0.5 punts) És cert que si a una xarxa de flux tots els arcs tenen capacitats amb valors diferents, el flux amb valor màxim és únic?

Solució: Fals. A la figura hi han 3 maneres diferents de definr un flux amb valor màxim.



(h) (0.5 punts) Què és un algorisme d'aproximació? Explica per què l'estratègia voraç, pot ser bona per a produir algorismes d'aproximació per a problemes que són NP-complets. Recordes algun exemple vist a classe?